

SVØMMEKLUBBEN DELFINEN

Administrativt Styringssystem

Projektrapport

Gruppe 5

Forfattere:

Carl Emil Zeidler

Nicklas Lærke Lyager

Tor Jørgensen Schønbech

Sulaman Shah

1. Semester · 2026

Afleveringsdato: 19. maj 2026

Vejledere: Cay Larsen & Jakob Janot

Indholdsfortegnelse

Indholdsfortegnelse	2
1. Produktvision.....	4
Systemet skal:.....	4
2. Liste over krav	5
2.1 Opdeling af krav (FURPS).....	5
Functionality (Funktionelle krav).....	5
Usability	5
Reliability	5
Performance	5
Supportability	5
3. User Stories	6
4. Aktører.....	12
Primære aktører	12
Acceptkriterier for User Stories.....	12
How to demo: "Beregn kontingent"	12
5. Domain Model	13
Centrale entiteter	13
6. Klassediagram	14
Pakkestruktur	14
Nøglerelationer	14
7. Hexagonal Arkitektur (Ports and Adapters).....	Error! Bookmark not defined.
De tre lag	Error! Bookmark not defined.
8. Scrum-processen	16
Scrum-roller	16
Product Backlog (prioriteret).....	16
Sprint 1 – Core MVP	18
Sprint Goal	18
Sprint Backlog.....	18
Sprint 1 Review	18
Sprint 2 – Konkurrencesystem.....	19
Sprint Goal	19
Sprint Backlog	19
Sprint 2 Review	19
Sprint Retrospective	21

Hvad gik godt?	21
Hvad kunne forbedres?	21
Definition of Done	22
9. Sekvensdiagrammer	23
9.1 Opret Medlem	23
9.2 Beregn Kontingent.....	24
9.3 Top 5 Svømmere pr. Disciplin	25
10. Dokumentation af Accepttests	27
Accepttest: Beregn Kontingent	27
Kode: ActiveMembership.calculateFee().....	27
Unit Tests.....	27
11. Risikoanalyse	29
12. Sammenhæng mellem Design og Kode	30
13. Konklusion	31
Kendte mangler	31

1. Produktvision

Formålet med systemet er at udvikle et administrativt system til Svømmeklubben Delfinen, som gør det lettere at håndtere medlemmer, kontingenter og konkurrencesvømmeres resultater.

Systemet hjælper klubbens formand, kasserer og trænere med deres daglige arbejde gennem en tekstbaseret løsning, der er brugervenlig, robust og let at vedligeholde.

Systemet skal:

- Registrere og administrere medlemmer (CRUD)
- Beregne medlemskontingenter automatisk baseret på alder og medlemskabstype
- Holde styr på betalinger og restancer
- Administrere konkurrencesvømmere og discipliner
- Registrere trænings- og konkurrenceresultater
- Vise top 5 svømmere inden for hver disciplin (junior/senior)
- Gemme og hente data fra filer (CSV)

Projektet er udviklet efter objektorienterede principper med fokus på ansvarlig klasseopdeling og lav kobling. Der anvendes hexagonal arkitektur (Ports and Adapters) for at adskille forretningslogik fra UI og persistence.

2. Liste over krav

2.1 Opdeling af krav (FURPS)

Functionality (Funktionelle krav)

- Registrere nye medlemmer med stamoplysninger (navn, alder, aktivitetsform)
- Oprette, redigere og slette medlemmer
- Konkurrencesvømmere inddeles i junior (under 18) og senior (18+)
- Tilknytte trænere til konkurrencesvømmere
- Registrere svømmediscipliner: Butterfly, Crawl, Rygcrawl, Brystsvømning
- Registrere bedste træningstid og dato pr. svømmer pr. disciplin
- Registrere konkurrenceresultater (stævne, placering, tid)
- Oversigt over top 5 svømmere pr. disciplin (opdelt i junior/senior)
- Beregne kontingenter automatisk (aktiv junior: 1.000 kr, aktiv senior: 1.600 kr, 60+: 25% rabat, passiv: 500 kr)
- Vise samlede forventede kontingentindtægter
- Vise liste over medlemmer i restance
- Gemme og indlæse data fra CSV-filer

Usability

- Tekstbaseret brugergrænseflade med klar menunavigation
- Systemet er tilgængeligt for formand, kasserer og træner via rollevalg

Reliability

- Systemet crasher ikke ved ugyldigt input (InputParser + InvalidInputException)
- Filhåndtering er robust med fejlhåndtering

Performance

- Hurtig søgning og sortering (Comparable/Comparator)

Supportability

- Let at udvide med nye discipliner (enum Discipline)
- Logisk opdeling i packages (domain, application, adapter)

3. User Stories

User stories følger formatet: As a <role>, I can <action>, so that <goal>

Aktør	User Story	Prioritet
Formand	Som formand vil jeg oprette, se, opdatere og slette medlemmer, for at administrere klubben.	P0 – Høj
Formand	Som formand vil jeg gruppere konkurrencesvømmere i hold baseret på alder, for at gøre det administrative lettere.	P0 – Høj
Formand	Som formand vil jeg tildele trænere til konkurrencesvømmere, for at sikre alle har en træner.	P1 – Medium
Kasserer	Som kasserer vil jeg have automatisk kontingentberegning, for korrekt opkrævning.	P0 – Høj
Kasserer	Som kasserer vil jeg se det samlede forventede kontingent, for overblik over klubbens indtægter.	P0 – Høj
Kasserer	Som kasserer vil jeg se hvem der er i restance, for at kontakte dem.	P0 – Høj
Træner	Som træner vil jeg se top 5 svømmere pr. disciplin (junior/senior), for udtagelse til stævner.	P1 – Medium
Træner	Som træner vil jeg se bedste træningstider for konkurrencesvømmere, for at se om de forbedrer sig.	P2 – Lav

Vores User storys skulle opfylde INVEST principperne:

- “I” ndependent (of all others)
- “N”egotiable (not a specific contract for features)
- “V”aluable
- “E”stimable (to a good approximation)
- “S”mall (so as to fit within an iteration)
- “T”estable (in principle, even if there isn’t a test for it yet)

For at finde ud af meningen med disse principper læste vi en [artikel](#), som diskuterer hvad manden bag INVEST, Bill Wake, mente med hvert princip. Vi stødte på følgende citat:

*“A story needs to be **valuable**. We don’t care about value to just anybody; it needs to be valuable to the customer. Developers may have (legitimate) concerns, but these framed in a way that makes the customer perceive them as important.*

This is especially an issue when splitting stories. Think of a whole story as a multi-layer cake, e.g., a network layer, a persistence layer, a logic layer, and a presentation layer

Dette ledte os til opdagelsen af Vertikale User Storys. For os gav det mening at Vertikale User Storys måtte være en konsekvens af INVEST principperne. Vi overvejede derfor om vi skulle anvende en form for Layered Architecture med elementer fra Clean Architecture såsom UseCase filer. UseCase filer korresponderede godt med hvad vi har lært om Use Cases. Vi indså også, at vi kunne skrive acceptance tests for hver Use Case, hvilket ville gøre vores tests meningsfulde og koblet til kravene i opgaven, og ikke tekniske detaljer som kunne variere efter refakturering og resultere i ufleksibel kode. Derfor valgte vi at arbejde med en sådan arkitektur. Selvom denne beslutning skulle vise sig at være en omfattende ambition, som har sat vores opgave bagud i forhold til de krav, som var i opgaven, så har det også været med til at sætte Scrum i et meningsfuldt perspektiv, og vi har lært meget af processen.

Vi har blandt andet erfaret, at de første User storys kan være svær at estimere tidsforbrug for, da det afhænger af rækkefølgen man implementerer dem. Derfor kan det være fornuftigt at starte med en såkaldt Sprint 0, hvor man implementerer det som kaldes en Walking Skeleton, som er en minimum fungerende end-to-end.

Arbejdet med Vertical User Storys end to end skulle dog vise sig i praksis at blive en udfordring, fordi der manglede tålmodighed for at gøre en story færdig før vi gik videre til den næste. I praksis blev det ofte sådan at vi arbejdede horizontalt mere end vertikalt, og gjorde aldrig nogle User Storys som Product Backlog Item helt færdigimplementeret. Det skyldes blandt andet at vi prioriterede at lade gruppemedlemmernes arbejde med de opgaver, som de følte de havde overskud og kompetencerne til at løse. Et område, som vi særligt manglede fokus på var tests. Dette gik også ud over vores Burndown chart, da vi havde som ambition at teste koden før vi ville markere den som ”færdig”. Resultatet blev desværre et helt fladt Burndown Chart. Som resultat af dette faldt vores motivation desværre også for at arbejde tæt med vores Scrum Board, for eksempel tog vi os ikke tid til at connecte pull requests med den korresponderende task.

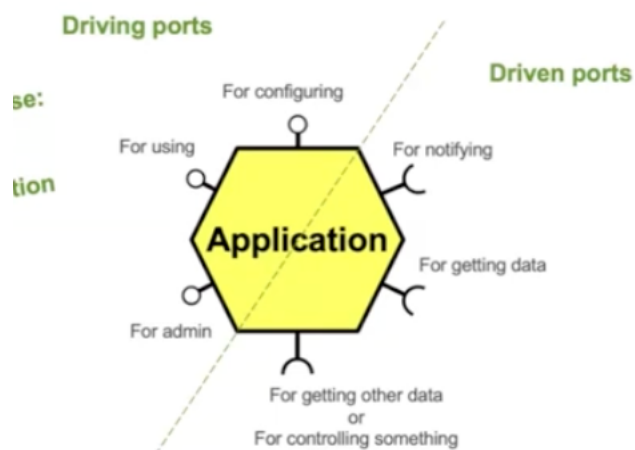
Ud over at være mere disciplineret og opmærksom på sådanne problemer, kunne en mulig løsning være, at arbejde mere med Test Driven Development. Vi gjorde os også erfaringer med denne arbejdsform, hvor vi skrev testen til InputParser før vi implementerede den. Dette virkede til at være en god måde også at samarbejde på, da en person kan skrive testen og en anden kan implementere den. På den måde sørger man for at der skabes en fælles forståelse af retningen for, hvor kodebasen skal hen og hvad som skal implementeres, hvilket kunne give mere mod på at gøre User Storys færdige, og samtidig tvinge dem til at være testbare.

3.1 Arkitektur

Hexagonal Arkitektur (Ports and Adapters)

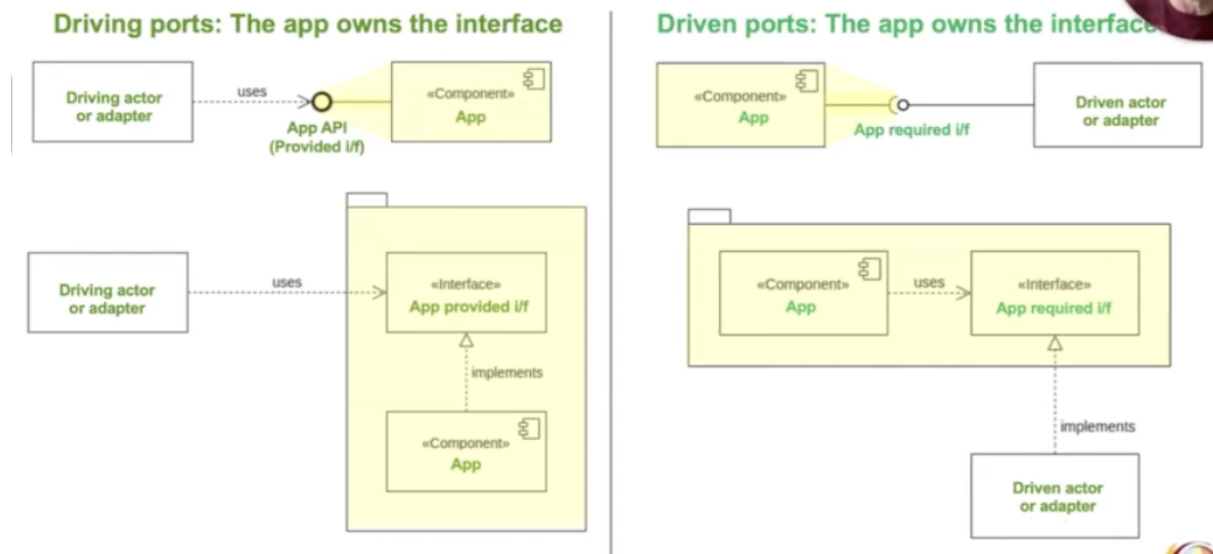
Hexagonal Arkitektur danner grundlaget for andre Layered Architectures, såsom Clean Architecture.

I Hexagonal Arkitektur snakker det ydre lag med de indre lag gennem interfaces kaldet Ports. De dele af det ydre lag som snakker med de indre lag kaldes Driving Adapters (også kaldet In, eller ofte Presentation), og det er dem som udløser koden, såsom brugerfladen. Driven Adapters (også kaldet Out, eller ofte Persistence) er de ydre lag, som reagerer på driving ports. Til forskel for Driving Adapters skal Driving ports ikke bruge den korresponderende port, men implementere den.



Figuren ovenover viser hexagonal arkitektur som et component diagram. Nedenunder vises forklaringen.

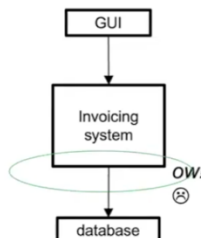
The source-code dependencies



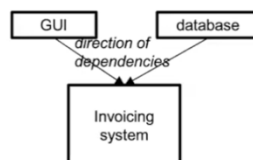
Fordelene ved hexagonal arkitektur er:

- Det er fleksibelt, da man kan skifte driven actors ud (for eksempel repositories, der hvor data gemmes, hvilket ofte er databaser), også mens koden kører. Dette opnås med dependency inversion ved [Strategy Pattern](#).

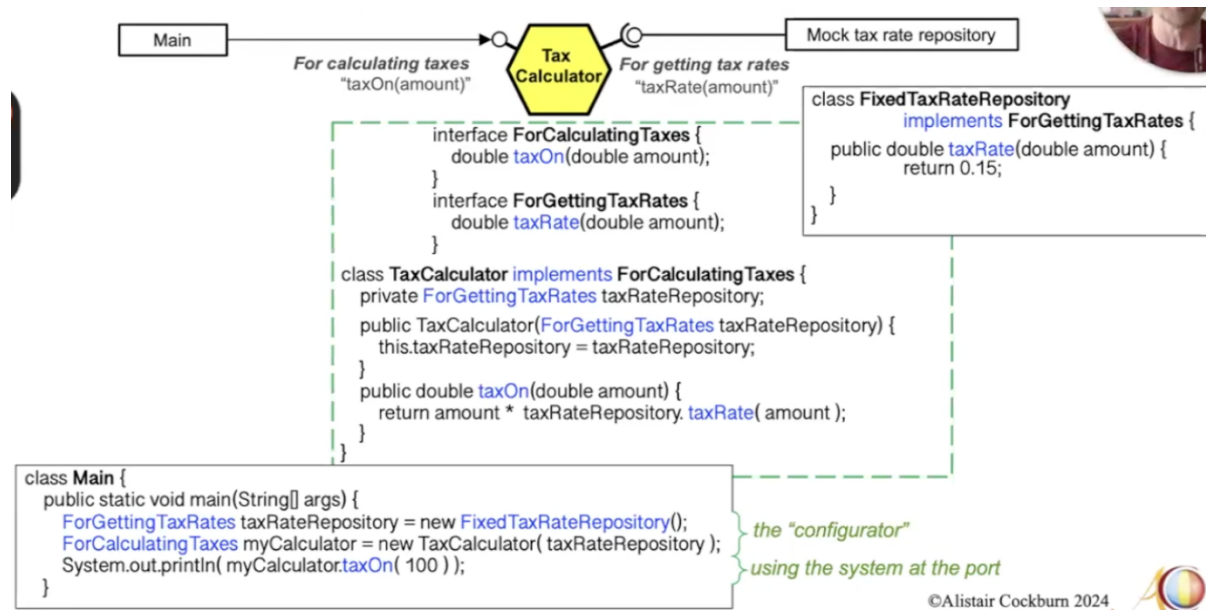
*Standard 3-layer dependencies
with difficulties*



*Preferred dependencies
easier to maintain & evolve*



- Man kan skifte driving adapters (UseCases, API's) ud med tests og driven adapters (Repositories, SPI's) ud med test doubles (mocks) på en måde, der er afkoblet fra kildekoden.
- Det begrænser forretningslogik til domænet, og adskiller det således fra brugerfladen.

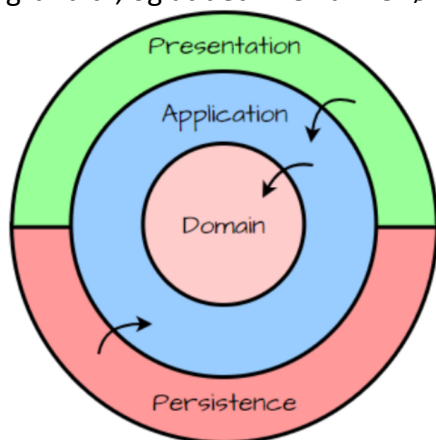


Figuren af Alistair Cockburn viser i en nøddeskal, hvordan Hexagonal Arkitektur opnås. Cockburn er manden bag dette mønster. ([link til hans video](#))

Ulempen er, at man skal injecte alt hvad Driving Adapters bruger i konstruktøren. Vores UseCases korresponderer ti Cockburns For- filer. Vi injektede altså alle UseCases! (vi forsøgte i hvert fald).

Clean Architecture(-ish)

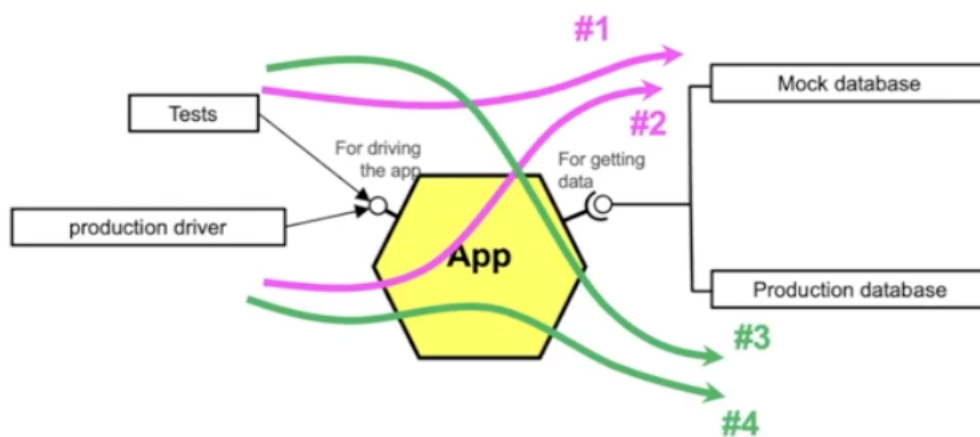
Den indre del af vores arkitektur er også opdelt i to lag, som er domæne, og application. Domain indeholder Entities, som er objekter med et id (såsom medlemmerne). Application indeholder orchestreringen af disse. Denne opdeling kommer vidstnok oprindeligt fra Onion Architecture. Pilene viser, hvilke lag der har en reference til hvilke lag. Vi overvejede, om Presentation måtte have en reference til Domain, og der kom vi frem til at det var der ingen grund til, og at det ville kunne føre til at de yderste lag anvendes til forretningslogik.



På figuren svarer Presentation til vores adapter.in package, altså driving adapters, og persistence svarer til vores adapter.out, altså driven adapters.

Vi testede, om disse pile var retvisende, og om de overholdt Hexagonal arkitektur. Til dette anvendte vi et testbibliotek der hedder **ArchUnit**. Se testen **HexagonalArchitectureTest.java**. Testen viser at vi ikke kom helt i mål med denne arkitektur.

Ideen med at lave UseCase filer, som kommer fra Clean Architecture, har blandt andet været at isolere kravene for at gøre dem testbare. Vi nåede ikke rigtigt at teste dem, men proceduren for integrationstests (eller acceptance tests) for disse ville så være i denne rækkefølge.



4. Aktører

Primære aktører

- Formand – Administrerer medlemsdata og konkurrencehold
- Kasserer – Administrerer økonomi og kontingentbetaling
- Træner – Administrerer svømmeresultater og holder øje med top 5

Acceptkriterier for User Stories

How to demo: "Beregn kontingent"

1. Log ind som kasserer
2. Vælg 'Se kontingenter' i menuen
3. Systemet beregner automatisk kontingent for hvert medlem
4. Kontrollér at junior (under 18) = 1.000 kr, senior = 1.600 kr, 60+ = 1.200 kr, passiv = 500 kr

5. Domain Model

Domain modellen identificerer de centrale konceptuelle klasser i systemet og deres relationer. Vi anvendte navneordsmetoden til at identificere disse, hvor vi highlightede alle navneord i opgavebeskrivelsen:

Centrale entiteter

Entitet	Attributter	Relationer
Member	id, navn, alder (fødselsår), aktivitetsform, membership, hasPaid, trainerName	Kan være CompetitiveSwimmer
CompetitiveSwimmer	member, trainer, disciplines (Liste), trainingResults, meetResults	Har 1 Trainer, mange Discipline og Results
Trainer	name	Tilknyttet til CompetitiveSwimmer
Discipline (enum)	Butterfly, Crawl, Rygcrawl, Brystsvømning	Tilhører CompetitiveSwimmer
TrainingResult	discipline, tid, dato	Tilhører CompetitiveSwimmer
MeetResult	discipline, tid, placering, stævne, dato	Tilhører CompetitiveSwimmer
Membership (interface)	calculateFee(age), getType()	Implementeret af ActiveMembership, PassiveMembership

6. Klassediagram

Systemet er opbygget med følgende klasser og deres relationer. Designet afspejler direkte koden der er implementeret:

Pakkestruktur

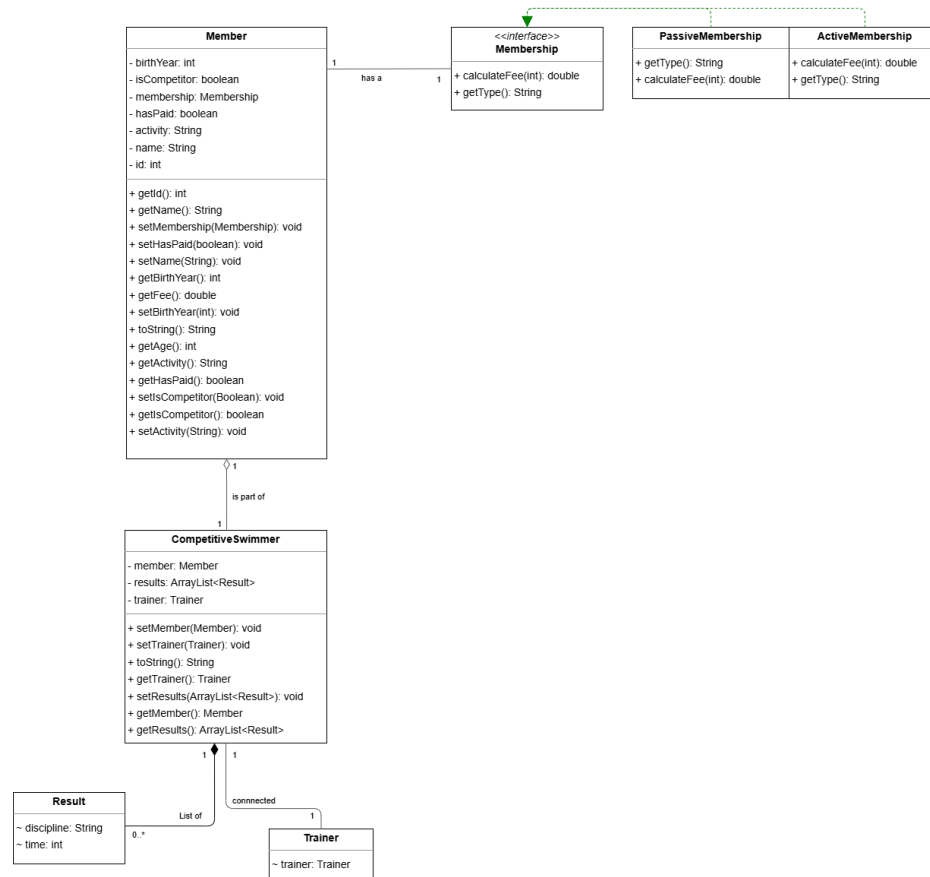
Package	Klasser/Interfaces
dk.delfinen.gruppe5.domain.model	Member, CompetitiveSwimmer, Trainer, Discipline (enum), TrainingResult, MeetResult
dk.delfinen.gruppe5.domain.service	Membership (interface), ActiveMembership, PassiveMembership, MemberService
dk.delfinen.gruppe5.application.port.in	RegisterMemberUseCase, DeleteMemberUseCase, EditMemberUseCase, CalculateMembershipFeeUseCase, SortMembersUseCase, Clock, ListMembersUseCase
dk.delfinen.gruppe5.application.port.out	MemberRepository (interface), IdGenerator (interface)
dk.delfinen.gruppe5.application.usecase	RegisterMemberUseCaseImpl, DeleteMemberUseCaseImpl, CalculateMembershipFeeUseCaseImpl, SortMembersUseCaseImpl, m.fl.
dk.delfinen.gruppe5.application.service	RandomIdGenerator, SystemClock
dk.delfinen.gruppe5.adapter.in	Controller, InputHandler, InputParser, Presenter, MenuOption, InvalidInputException
dk.delfinen.gruppe5.adapter.out.persistence	FileMemberRepository, FileMemberSerializer, InMemoryMemberRepository

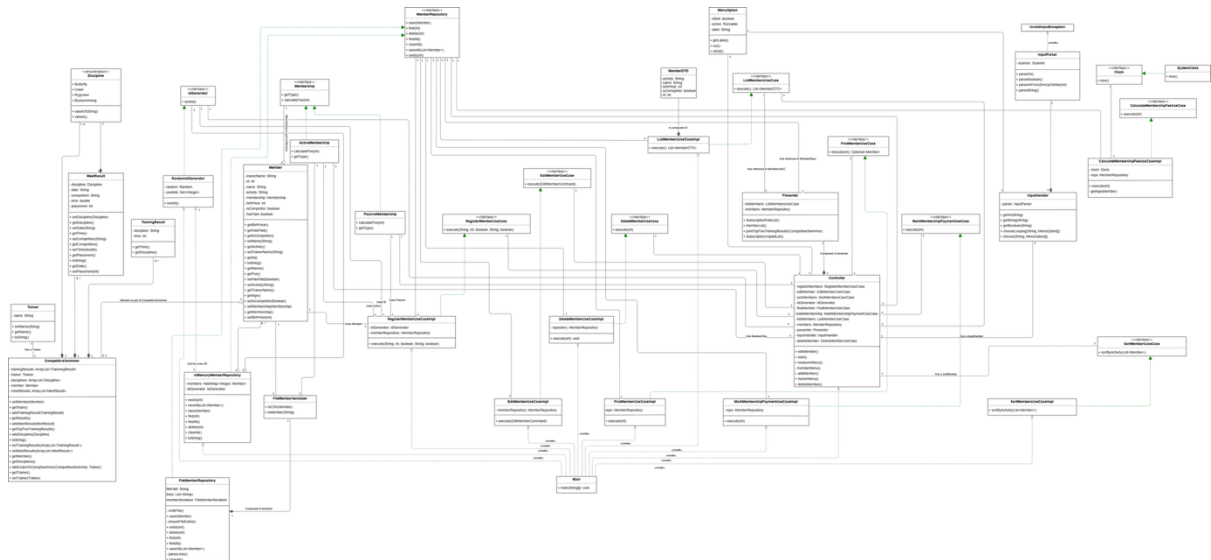
Nøglerelationer

- Member er basisklasse – CompetitiveSwimmer wrapper en Member (komposition, ikke arv).
- Membership er et interface – implementeret af ActiveMembership og PassiveMembership (Strategy Pattern)
- MemberRepository er et interface – implementeret af FileMemberRepository og InMemoryMemberRepository
- UseCase-interfaces i port/in definerer hvad adapteren må kalde

Under Projektet lavede vi 2 Klasse diagrammer, et tidligere i processen og et af det færdige projekt.

Det første er over det som vi havde lavet på domain pakken, så ting som Member, Trainer og CompetitiveSwimmer. Det diagram gav gruppen et visuelt overblik over hvordan koden så ud, og hvordan vi kunne arbejde videre med den.





Dette er et klasse diagram af næsten hele vores kode, den giver et visuelt fælles indblik i hvordan det hele er forbundet, på en måde man ikke kan i sit eget hoved.

Link til det store klasse diagram.

<https://drive.google.com/file/d/1B8LdnAn0Wd4XK6MI8ZCwOsyGYGTp3gO-/view?usp=sharing>

8. Scrum-processen

Scrum-roller

- Product Owner: Niklas (repræsenterer svømmeklubben, og har ansvar defineret i Scrum Guiden såsom prioritering af User Storys)
- Scrum Master: Carl Emil (facilitator, sikrer derfor blandt andet at Scrum arbejdsmodellen følges).

”Det har været en udfordring at være Scrum master, da jeg nogle gange har mistet overblikket over Scrum Boardet. Selvom jeg oprettede Scrum boardet og således faciliterede denne del, følte jeg ikke at jeg fik et overblik over udformningen af tasks, hvilket skyldtes problemer med at oprette et clean Github Repository, som tog en hel dag, sammen med stress af personlige grunde. Vi har holdt daily scrums, og sprint review og retrospective, der har dog ikke været tid til sprint review og sprint retrospective i sprint to”

- Development Team: Hele gruppen (alle udviklere, herunder Scrum Master og Product Owner, som har haft dobbeltroller)

Product Backlog (prioriteret)

ID	User Story	Points	Prioritet
1	Opret medlem	5	P0
2	Rediger medlem	5	P0
3	Slet medlem	3	P0
4	Beregn kontingent	5	P0
5	Se restancer	3	P0
6	Samlet kontingent	3	P0
7	Gem data i fil	8	P0
8	Indlæs data fra fil	8	P0
9	Filtrer aktive/passive	3	P1
10	CompetitiveSwimmer	5	P1
11	Tildel træner	5	P1
12	Discipliner (enum)	3	P1
13	Registrer resultater	8	P1
14	Top 5 svømmere	13	P1
15	Træneroversigt	3	P2

Estimering sker med Planning Poker og Fibonacci-skalaen (1, 3, 5, 8, 13). Eksempel: 'Vis top 5 svømmere' fik 13 points pga. kompleks sortering, filtrering og opdeling i junior/senior.

Sprint 1 – Core MVP

Sprint Goal

Bygge de grundlæggende datastrukturer og et MVP-system som resten kan bygges på.

Sprint Backlog

- Opret Member-klasse med constructor og getters/setters
- Implementer Membership-interface + ActiveMembership + PassiveMembership
- Lav Controller med tekstbaseret UI-flow
- Implementer calculateFee() med alle regler
- Lav FileHandler (FileMemberRepository + FileMemberSerializer)
- Implementer save() og load() til CSV

Sprint 1 Review

Vi havde oprettet et burndown chart men det giver ikke mening at vise da vi ikke fik færdigimplementeret nogle tasks i og med at vi ikke testede noget.

Færdigt: Opret, rediger, slet medlem. Beregn kontingent. Gem/indlæs filer.

Ikke færdigt: UI-forbedringer. Filhåndtering krævede ekstra tid.

Feedback: Systemet fungerer stabilt.

Sprint 1 Retrospective

Vi besluttede at lave en mere uformel form for risikoanalyse, som vi viser herunder.

Risikotanalyse				
Risikomoment	Konsekvens	Prævention Tiltag	Hvorfor skete det?	Løsningsforslag
P-1 Manglende test	Koden kan i visse scenarier opføre sig uhensigtsmæssigt i nogle scenarier	Vi starter med at teste alle use cases, også før vi har implementeret dem (Test Driven Developement)		
P-2 Github permission problemer	Masser af tid brugt på at fixe permissions	Test at alle kan push og pull, så det ikke bliver en overraskelse.		

Risikotanalyse				
P-3 Merging katastrofe	Masse tidsspilde med at manuelt skulle merge koden	Husk at pull før du starter en ny branch.	Folk glemmer at pull før de starter på det næste	Før du starter på en ny task, kig på git om der ligger.
P-4 ved ikke hvad jeg skal gå i gang med				
P-5 Det er svært at designe domænet		Lav klassediagram, udvikling ud fra use case tests måske.		

Sprint 2 – Konkurrencesystem

Sprint Goal

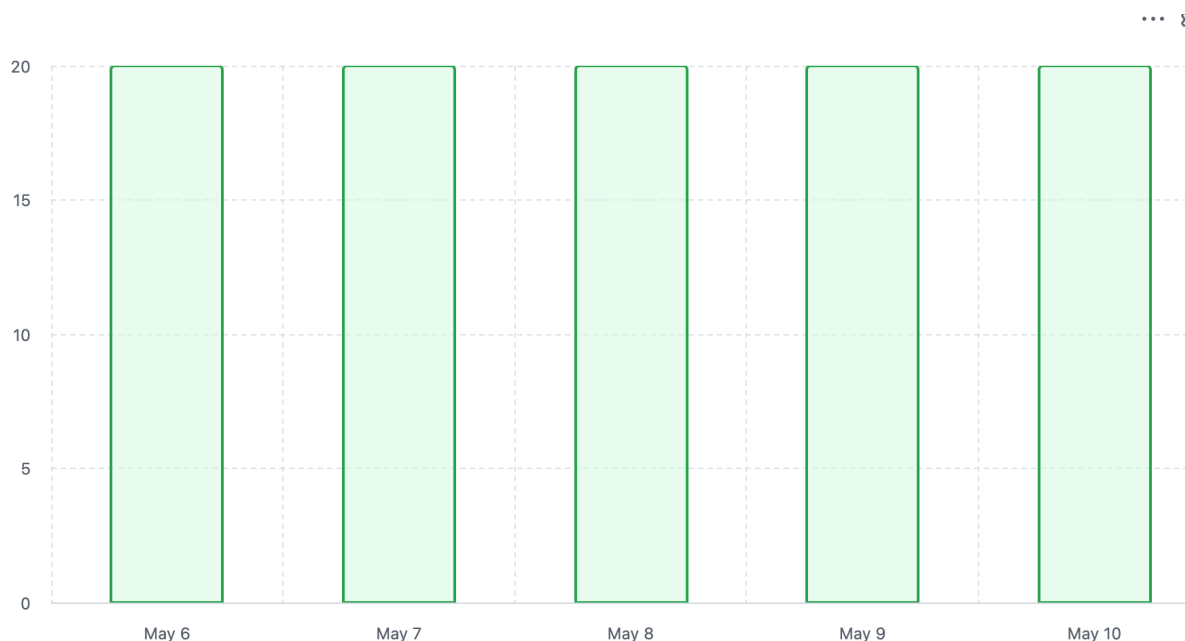
Blive færdig med alle kriterierne i opgaven. Klar til fremlæggelse.

Sprint Backlog

- Lav CompetitiveSwimmer (wrapper om Member)
- Lav Trainer-klasse og tilknyt til svømmere
- Opret Discipline-enum med 4 discipliner
- Lav TrainingResult og MeetResult klasser
- Implementer top 5 sortering med Comparator
- Kasserer: restancer, samlet kontingent

Sprint 2 Review

Igen, Burndown chartet giver ikke mening, da vi ikke har testet meget. Dette er det mest retvisende billede (jo der mangler ideallinje, men giver det mening at tegne den?).



Dog kan vi se at mange features kan demonstreres, så det er ikke fordi vi ikke har lavet noget.

1	▼	🕒 Vis alle Medlemmer #56	Sprint Backlog	▼
		🕒 Vis medlemmer #31	Ready to test	▼
2	▼	🕒 se mine svømmere (Træner) #11	Sprint Backlog	▼
		🕒 Lav en "trainers" ArrayList inde i competitiveSwimmer klassen #55	In progress	▼
3	▼	🕒 Status på betalte medlemmer #7	Sprint Backlog	▼
		🕒 Lav "harBetal" felt til kontingens klassen #40	Ready to test	▼
		🕒 Opdel i hvem der har betalt og hvem der ik har #42	Ready to test	▼
		🕒 Print en oversigt over hvem der har betalt og hvem der mang... #43	Ready to test	▼
4	▼	🕒 Udregning af kontigent #6	Sprint Backlog	▼
		🕒 Lav kontingent klasse #33	Ready to test	▼
		🕒 Lav felterne "passiv" "ungdom" "senior" og "rabat" #34	In progress	▼
		🕒 Lav en metode til at beregne kontingens #36	Ready to test	▼
		🕒 Lav en metode til at udskrive kontingent #37	Ready to test	▼
		🕒 Lav en metode til beregning af det samlede kontingent #38	Ready to test	▼

5	▼	🕒 Håndtere medlemmer #2	Sprint Backlog	▼
		🕒 lav en medlem klasse #12	Ready to test	▼
		🕒 tilføj medlem #13	Ready to test	▼
		🕒 Slet medlem #14	Ready to test	▼
		🕒 Redigering af Medlem #30	Ready to test	▼
	▼	🕒 Opret klasse competitiveSwimmer #18	Ready to test	▼
		🕒 Lav array list af forskellige discipliner #19	In progress	▼
		🕒 Tilføj svømmedisciplin til CompetitiveSwimmer #44	In progress	▼
		🕒 Membership #60	Ready to test	▼
6	▼	🕒 Top 5 svømmere #9	Product Backlog	▼
		🕒 Registrering af stævneresultater #53	Sprint Backlog	▼
		🕒 Top 5 svømmer pr. disciplin #54	In progress	▼
7	▼	🕒 Træningstider #10	Product Backlog	▼
		🕒 Lav TrainingResult klasse #48	Sprint Backlog	▼
		🕒 Lav en ArrayList over svømmerene, deres diciplin, deres tid o... #46	In progress	▼
		🕒 Lav metode til at registrere træningstid #51	Ready to test	▼
		🕒 Sorter så vi kan se de fem bedste svømmere #47	In progress	▼
		🕒 lav metode til at udskrive bedste tider #52	Ready to test	▼
8	▼	🕒 Træner tildeling til Konkurrence Svømmer #58	Product Backlog	▼
		🕒 Tildeling af træner #50	Ready to test	▼

Færdigt: CompetitiveSwimmer, Trainer, Resultater, Top 5, Træneroversigt, Restancer.

Feedback: Sortering fungerede godt. Domain model blev udvidet undervejs.

Sprint Retrospective

Hvad gik godt?

- GitHub branches fungerede bedre i Sprint 2
- Gruppen samarbejdede godt
- Hexagonal arkitektur gav god struktur

Hvad kunne forbedres?

- Mere test tidligt i processen
- Bedre merge-strategi (husk at pull inden ny branch)

- Mindre tid på debugging – mere TDD

Definition of Done

Følgende har været vores definition af hvornår vi er færdige.

En user story anses som færdig når:

- Funktionelle krav er opfyldt
- Koden er testet
- Acceptkriterier er opfyldt
- Koden er integreret i main branch
- Funktionaliteten kan demonstreres
- Koden følger projektets arkitektur

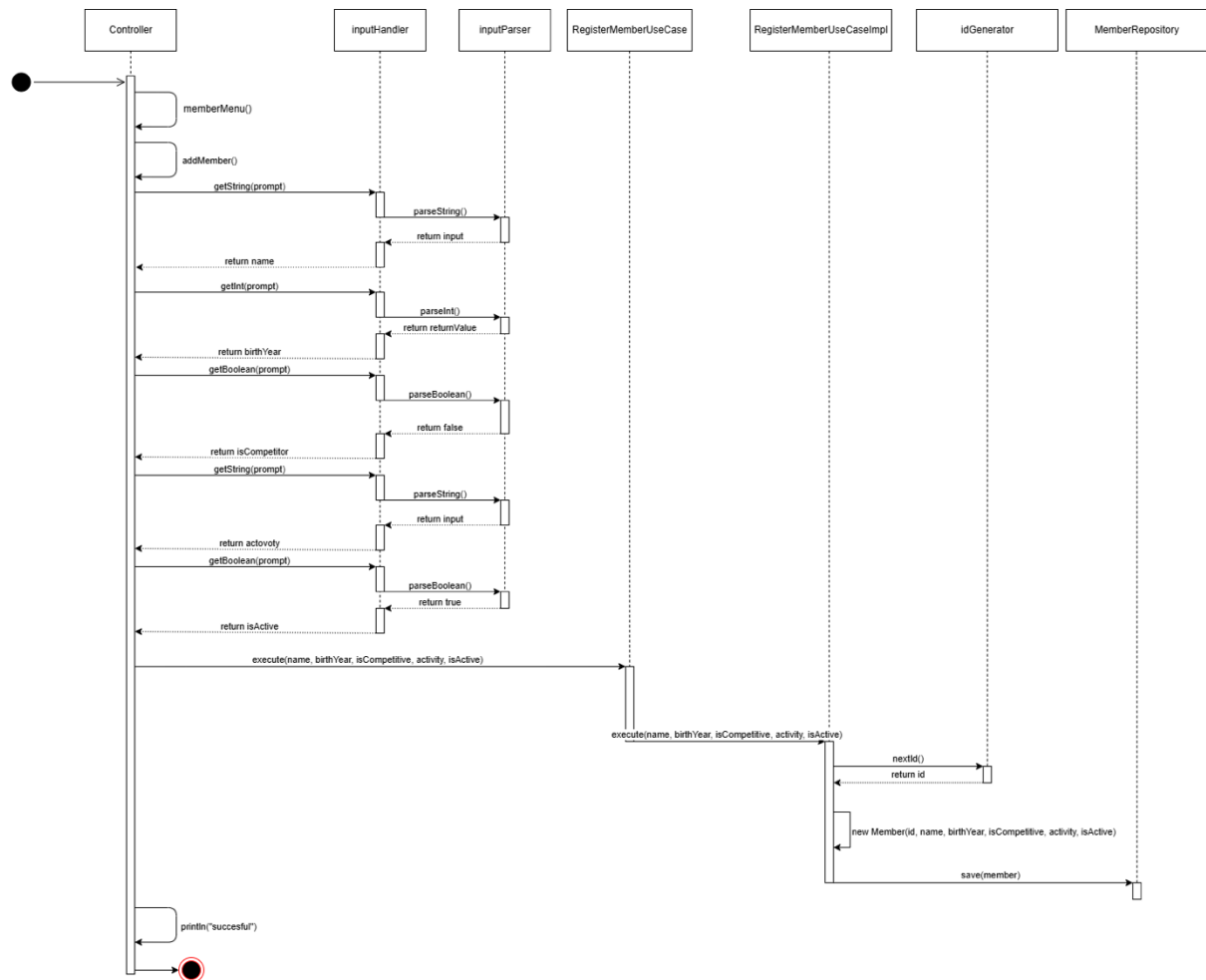
Vi kom ikke i mål, og det kunne have hjulpet os ikke at fokusere på en delmængde af User storys, før vi gik i gang med den næste.

9. Sekvensdiagrammer

9.1 Opret Medlem

Aktør: Formand · Beskrivelse: Formanden opretter et nyt medlem i systemet.

#	Fra → Til	Besked/Handling
1	Controller → Controller	Printer de forskellige muligheder man kan vælge
2	Controller → Controller	“addMember” starter processen til at tilføje et medlem
3	Controller → inpuhandler	Bruger scanner til at få et input fra brugeren
4	inpuhandler → inputParser	Sender inputtet vider og tjekker om det er lovligt
5	InputParser → inputHandler	Hvis inputtet er lovligt sendes det tilbage til indpuHandler
6	inputHandler → Controller	Returnere hvad det blev inputtet i form af name, birthYear, isCompetitor, activity og isActive.
7	Controller → RegisterMemberUseCase	Alle variablerne den nye member består af bliver sendt videre til UseCase for RegisterMember
8	RegisterMemberUseCase → RegisterMemberUseCaseImpl	UseCase sender den så videre til RegisterMemberUseCaseImpl, det gøres for at holde de forskellige lag adskilt.
9	RegisterMemberUseCaseImpl → idGenerator	Der bliver kaldt på idGenerator for at skabe et nyt id til memberen vi laver, det gøres for at være sikker på at vært id er unikt
10	RegisterMemberUseCaseImpl → RegisterMemberUseCaseImpl	Da alt den relevante data nu er indsamlet kan de kombineres til at skabe en ny member
11	RegisterMemberUseCaseImpl → MemberRepository	Da memberen nu er skabt blive han gemt i member.csv filen



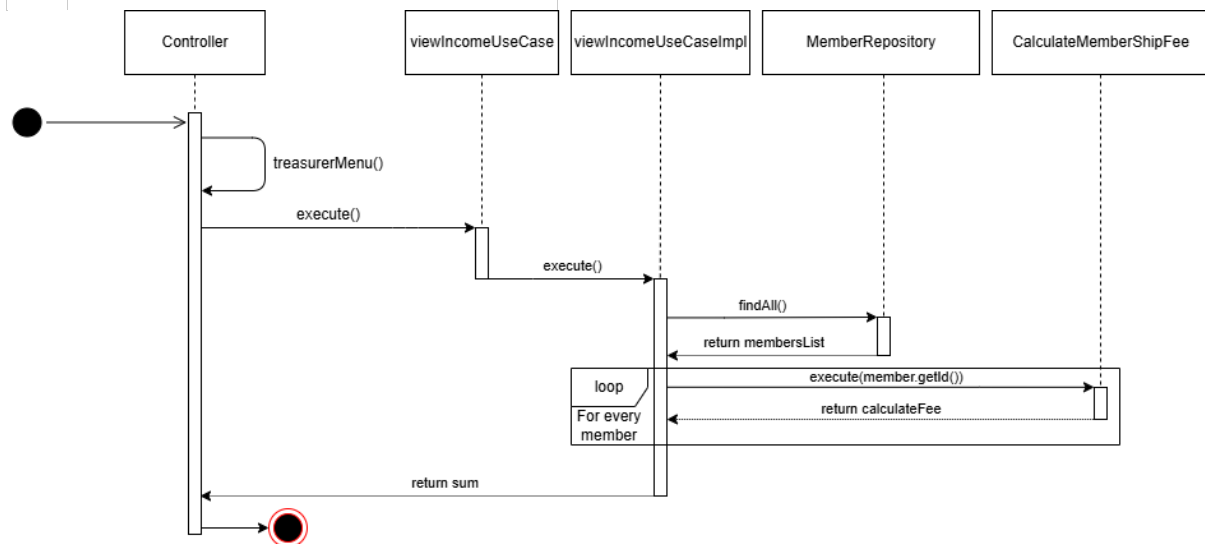
Stikord:

- RegisterMemberUseCaseImpl opretter et Member-objekt og gemmer det i Repository, som står for fil- eller datalagring.
- Til sidst sendes en succesbesked tilbage til brugeren.

9.2 Beregn Kontingent

Aktør: Kasserer · Beskrivelse: Systemet beregner automatisk kontingent for et givet medlem.

#	Fra → Til	Besked/Handling
1	User → Controller	Vælger 'se kontingent' fra kasserer-menuen
2	Controller → CalculateMembershipFeeUseCaseImpl	execute(memberId)
3	UseCase → MemberRepository	find(id) → henter Member-objekt
4	UseCase → Clock	now().getYear() → henter aktuelt år
5	UseCase → member.getMembership()	calculateFee(age) → ActiveMembership eller PassiveMembership
6	ActiveMembership	Returnerer 1000/1600/1200 kr baseret på alder
7	Controller → Presenter	Viser beregnet kontingent til brugeren



Stikord:

- Kassereren vælger funktionen i UI'et, hvorefter controlleren kalder CalculateMembershipFeeUseCase.
- Systemet beregner derefter kontingentet ud fra forretningsreglerne i opgaven, såsom alder, aktiv/passiv status og 60+ rabat.
- Resultatet returneres til controlleren og vises derefter i UI'et.

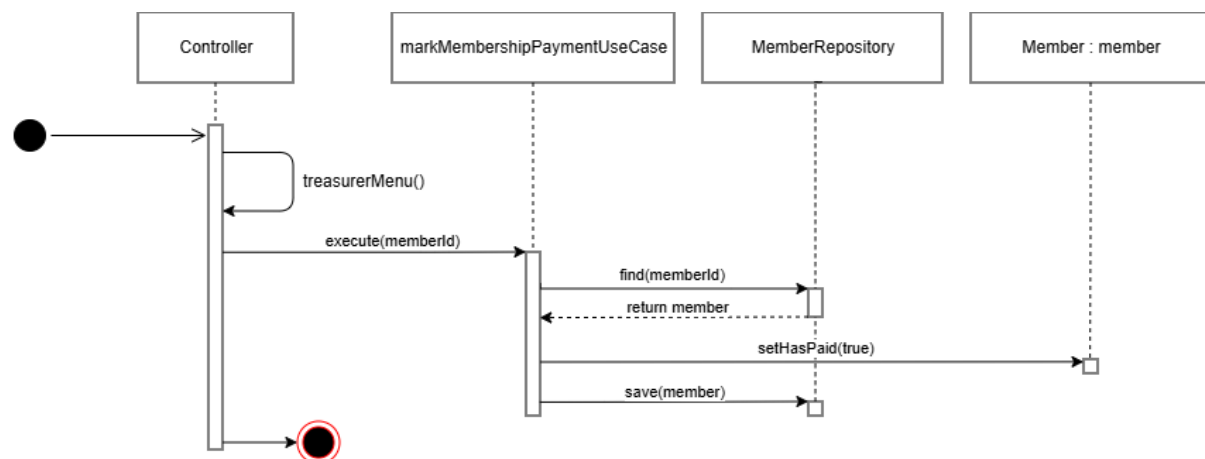
Diagrammet viser hvordan forretningslogikken er isoleret i en use case-klasse, hvilket gør systemet mere vedligeholdelsesvenligt til næste programmør.

9.3 Top 5 Svømmere pr. Disciplin

Aktør: Kasserer · Beskrivelse: Kasserer markerer en member som betalt.

#	Fra → Til	Besked/Handling
1	Controller →	Udprinter alle de muligheder kasserer har

	Controller	
2	Controller → MembershipPayment	Kasseren har på baggrund af member Id valgt hvilket member som skal markers som HasPaid
3	MembershipPayment → MemberRepository	Idet som kasseren har givet bliver brugt til at finde det spicifikke medlem som der skal markeres
4	MemberRepository → MembershipPayment	Efter den har fundet memberen returneres den
5	MembershipPayment → Member : member	Memberen som blev fundet med Id'et bliver market som HasPaid = true.
6	MembershipPayment → MemberRepository	Efter at memberen er markeret som HasPaid = true, så bliver memberens info i MemberRepository opdateret.



10. Dokumentation af Accepttests

Accepttest: Beregn Kontingent

Tester at kontingentberegning (ActiveMembership.calculateFee()) opfylder kravene:

Test	Input	Forventet	Resultat
1	Aktivt junior-medlem, alder 16 år	1.000 kr/år	PASS
2	Aktivt senior-medlem, alder 25 år	1.600 kr/år	PASS
3	Aktivt senior-medlem, alder 65 år	1.200 kr/år (25% rabat)	PASS
4	Passivt medlem (hvilken som helst alder)	500 kr/år	PASS

Kode: ActiveMembership.calculateFee()

```
public double calculateFee(int age) {
    if (age < 18) return 1000;           // Junior
    double fee = 1600;                   // Senior
    if (age >= 60) fee *= 0.75;          // 25% rabat (60+)
    return fee;
}
```

Unit Tests

Testfil	Hvad testes
MemberTest.java	getId, getName, getBirthYear, getAge, getFee, toString
CompetitiveSwimmerTest.java	Discipliner, træner-tildeling, resultater
InputParserTest.java	Parsning og validering af brugerinput
RegisterMemberUseCaseImplTest.java	Oprettelse af nyt medlem via use case
HexagonalArchitectureTest.java	Bekræfter at lagdeling respekteres (ArchUnit)
MemberCRUDAcceptanceTest.java	Full CRUD acceptance flow

Unit Tests

For at sikre korrekt funktionalitet i systemet er der udviklet unit tests til de vigtigste dele af forretningslogikken.

Der er især fokus på kontingentberegning, da denne funktion indeholder flere regler afhængigt af alder og medlemskabstype.

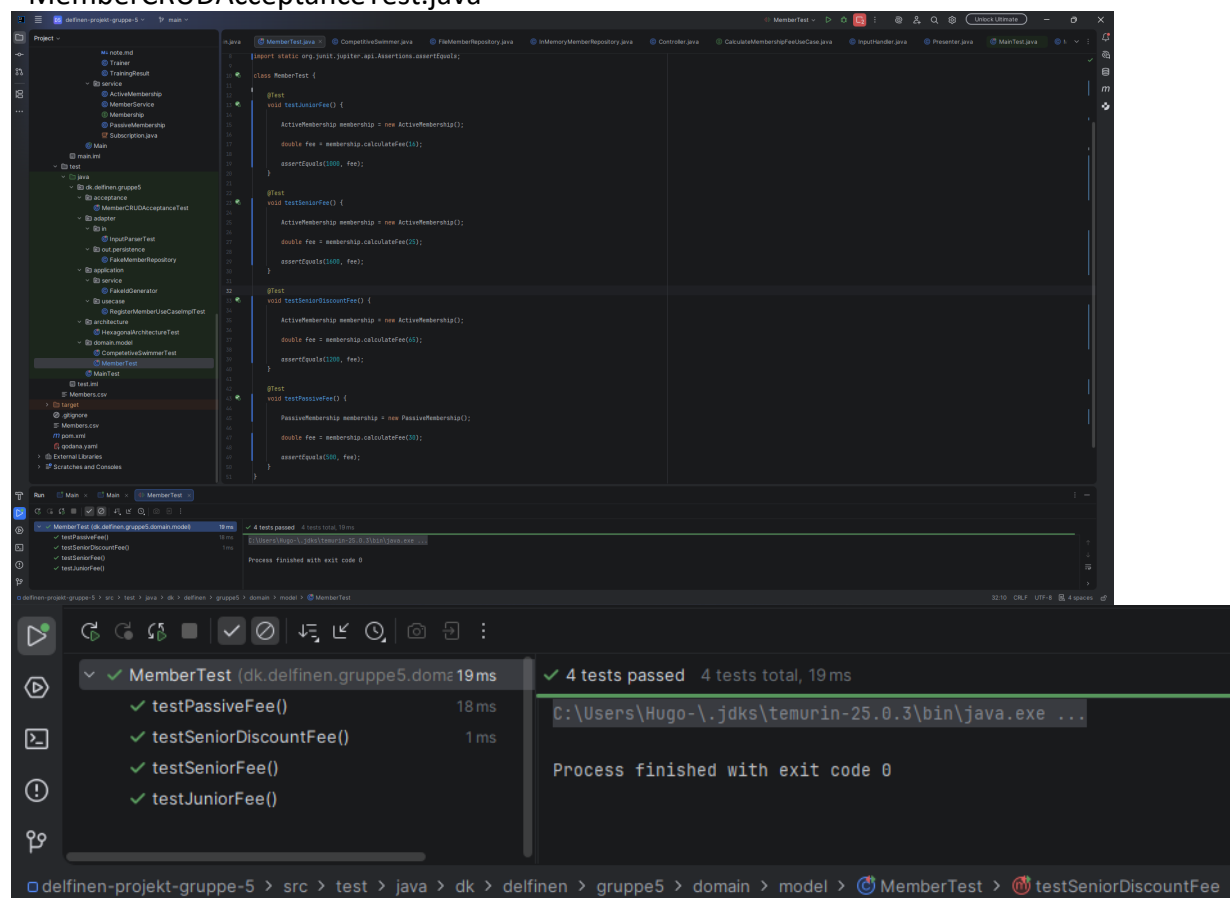
Følgende scenarier er testet:

Test	Input	Forventet resultat
---	---	---
Aktiv junior	16 år	1000 kr
Aktiv senior	25 år	1600 kr
Senior over 60 år	65 år	1200 kr
Passivt medlem	Passivt medlemskab	500 kr

Der er anvendt JUnit 5 til testene da den automatisk tester vores kode. Testene verificerer at systemets beregninger følger kravene fra casen.

Testfiler:

- MemberTest.java
- CompetitiveSwimmerTest.java
- InputParserTest.java
- RegisterMemberUseCaseImplTest.java
- HexagonalArchitectureTest.java
- MemberCRUDAcceptanceTest.java



11. Risikoanalyse

ID	Risiko	Konsekvens	Prævention	Status
P-1	Manglende test	Fejl i edge cases	TDD – test before implement	Delvist løst
P-2	GitHub permissions	Tidsspilde	Test push/pull tidligt	Løst
P-3	Merge-konflikter	Tidsspilde, datatab	Pull før ny branch	Delvist løst
P-4	Uklar opgavefordeling	Dobbeltarbejde	Daily Scrum og Sprint Planning	Løst
P-5	Kompleks domain model	Forkert design	Navneordsanalyse + iterativ tilgang	Løst

12. Sammenhæng mellem Design og Kode

Der er direkte sammenhæng mellem domain model, klassediagram og den implementerede kode:

Design-element	Implementering i kode
Member (domain model)	Member.java i domain/model – indeholder alle stamoplysninger
CompetitiveSwimmer (specialisering)	CompetitiveSwimmer.java – wrapper Member, tilføjer trainer, disciplines, results
Membership (interface)	Membership.java interface → ActiveMembership.java, PassiveMembership.java
Disciplin (enum)	Discipline.java enum med Butterfly, Crawl, Rygcrawl, Brystsvømning
Resultat	TrainingResult.java og MeetResult.java med tid, dato, placering
Kontingentberegning	calculateFee(int age) i Membership-interface, implementeret i Active/PassiveMembership
UseCase (use case diagram)	UseCase-interfaces i port/in, implementeret i application/usecase
Repository (data)	MemberRepository interface → FileMemberRepository (CSV) + InMemory

13. Konklusion

Projektet har resulteret i et fuldt fungerende administrativt system for Svømmeklubben Delfinen. Systemet lever op til de fleste krav i opgavebeskrivelsen:

- Alle CRUD-operationer for medlemmer er implementeret og fungerer
- Kontingentberegning er korrekt implementeret med alle regler (junior, senior, 60+, passiv)
- Hexagonal arkitektur er gennemført med klar adskillelse af lag
- CSV filhåndtering gemmer og indlæser data korrekt
- Unit tests og accepttests er implementeret
- Scrum-processen er fulgt med 2 sprints, daily scrum, reviews og retrospectives

Kendte mangler

- Træner-menuen er ikke fuldt implementeret i Controller
- Main.java indeholder test-kode der burde fjernes
- Subscription.java er commented out og bør ryddes op
- Da vi havde travlt til sidst, endte vi med at ikke kunde færdig gøre Klasse Diagrammet med den kode som kom sidste minut. Eller lave en gruppe gennemgang.

Disse mangler er identificeret og ville være næste sprint's fokus ved videreudvikling. Projektet har givet god erfaring med OOP, hexagonal arkitektur, Scrum og Java-programmering.